Name: Nguyễn Dương Phúc
Student ID: 24521386
Class: CS4323.Q12.CTTT

OS
LAB 5

CHECKLIST

1. Homework

	1	2	3
Explanation	☒	☒	☒
Your code	☒	☒	☒
Results and discussions	☒	☒	☒

Mark: 10/10 (self-scored)

**Notice: Export PDF file, name format:*
<MSSV>_LAB5.pdf

2. Lab 5

1. Implement the consume and produce

sells <= products <= sells + <4 last digit of SID>

Answer:

Step 1: Create the source file

```
yangpuck@NguyenDuongPhuc:~$ cat > exercise1.cpp
```

Step 2: C++ implementation (pthread + semaphore)

Open ▾ *exercise1.cpp ~/ Save

```cpp
#include <iostream>
#include <pthread.h>
#include <unistd.h>

using namespace std;

int sells = 0;
int products = 0;
int s_id = 1135;       // 4 số cuối SID
bool stop = false;

pthread_mutex_t lock;

void* sellThread(void*);
void* productThread(void*);

int main() {
    pthread_t t_tid[2];

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t_tid[0], NULL, sellThread, NULL);
    pthread_create(&t_tid[1], NULL, productThread, NULL);

    pthread_join(t_tid[0], NULL);
    pthread_join(t_tid[1], NULL);

    pthread_mutex_destroy(&lock);
```

C++ ▾ Tab Width: 8 ▾ Ln 68, Col 2 ▾ INS

```
    cout << "Chuong trinh ket thuc." << endl;
    return 0;
}

void* sellThread(void* arg) {
    while (!stop) {
        pthread_mutex_lock(&lock);

        // sells <= products
        if (sells < products) {
            sells++;
            cout << "Sells: " << sells << endl;
        }

        if (sells >= 20) {
            stop = true;
        }

        pthread_mutex_unlock(&lock);
        sleep(1);
    }
    pthread_exit(NULL);
}

void* productThread(void* arg) {
    while (!stop) {
        pthread_mutex_lock(&lock);

        // products <= sells + s_id
        if (products < sells + s_id) {
            products++;
            cout << "Products: " << products << endl;
        }

        pthread_mutex_unlock(&lock);
        sleep(1);
    }
    pthread_exit(NULL);
}
```

Explanation:

- The program uses **two threads**:
  - productThread (Producer)
  - sellThread (Consumer)
- products represents the number of produced items.
- sells represents the number of consumed items.
- The producer increases products only if:

products < sells + SID

- The consumer increases sells only if:

sells < products

- A **mutex lock** is used to protect shared variables and prevent race conditions.
- The program stops when sells reaches 20.
- The sleep() function is used to avoid busy waiting and reduce CPU usage.

Step 3: Compile the program

```
yangpuck@NguyenDuongPhuc:~$ g++ exercise1.cpp -o exercise1 -pthread
```

Step 4: Run the program and observe the output

```
yangpuck@NguyenDuongPhuc:~$ ./exercise1
Products: 1
Sells: 1
Products: 2
Sells: 2
Products: 3
Sells: 3
Products: 4
Products: 5
Sells: 4
Products: 6
Sells: 5
Products: 7
Sells: 6
Products: 8
Sells: 7
Products: 9
Sells: 8
Sells: 9
Products: 10
Sells: 10
Products: 11
Sells: 11
Products: 12
```

```
Sells: 12
Products: 13
Products: 14
Sells: 13
Sells: 14
Products: 15
Sells: 15
Products: 16
Products: 17
Sells: 16
Products: 18
Sells: 17
Sells: 18
Products: 19
Sells: 19
Products: 20
Products: 21
Sells: 20
Chuong trinh ket thuc.
```

2. Given 4 threads T1, T2, T3, T4, whose jobs are print their name, running simultaneously. Let's implement the process/thread synchronization for these 4 threads using the diagram beside.

Answer:

Step 1: Create the source file

```
yangpuck@NguyenDuongPhuc:~$ cat > exercise2.cpp
```

Step 2: C++ implementation (pthread + semaphore)

Open ▾ *exercise2.cpp ~/

```
#include <iostream>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

using namespace std;

sem_t sem_t1;
sem_t sem_t2t3;

void* T1(void* arg) {
    cout << "T1 is running" << endl;
    sleep(1);

    sem_post(&sem_t1);
    sem_post(&sem_t1);   // allow T2 and T3
    pthread_exit(NULL);
}

void* T2(void* arg) {
    sem_wait(&sem_t1);
    cout << "T2 is running" << endl;
    sleep(1);

    sem_post(&sem_t2t3);
    pthread_exit(NULL);
}
```

```
void* T3(void* arg) {
    sem_wait(&sem_t1);
    cout << "T3 is running" << endl;
    sleep(1);

    sem_post(&sem_t2t3);
    pthread_exit(NULL);
}

void* T4(void* arg) {
    sem_wait(&sem_t2t3);
    sem_wait(&sem_t2t3);   // wait for both T2 and T3

    cout << "T4 is running" << endl;
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2, t3, t4;

    sem_init(&sem_t1, 0, 0);
    sem_init(&sem_t2t3, 0, 0);

    pthread_create(&t1, NULL, T1, NULL);
    pthread_create(&t2, NULL, T2, NULL);
    pthread_create(&t3, NULL, T3, NULL);
    pthread_create(&t4, NULL, T4, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    sem_destroy(&sem_t1);
    sem_destroy(&sem_t2t3);

    return 0;
}
```

Explanation:

- Four threads are created and run concurrently.
- **Semaphore sem_t1** ensures that T2 and T3 start only after T1 finishes.
- **Semaphore sem_t2t3** ensures that T4 starts only after both T2 and T3 finish.
- T2 and T3 signal completion independently.

- T4 waits for two signals before executing.
- Semaphores are used to enforce the required execution order.

Step 3: Compile the program

```
yangpuck@NguyenDuongPhuc:~$ g++ exercise2.cpp -o exercise2 -pthread
```

Step 4: Run the program and observe the output

```
yangpuck@NguyenDuongPhuc:~$ ./exercise2
T1 is running
T2 is running
T3 is running
T4 is running
```

3. Given an array a declared as an integer array with n elements, a is declared as a global variable. Write a program that includes 2 threads running concurrently:

Thread 1: performs the task of generating random integers, then adding them to array a. After adding, it counts and outputs the number of elements in a immediately after insertion

Thread 2: takes an element from array a (any element, depending on the implementation), then counts and outputs the number of elements in a immediately after removal. If there are no elements left in a, it should display "Nothing in array a".

Answer:

Step 1: Create the source file

```
yangpuck@NguyenDuongPhuc:~$ cat > exercise3.cpp
```

Step 2: C++ implementation (pthread + semaphore)

Open ▾ *exercise3.cpp
~/

```cpp
#include <iostream>
#include <pthread.h>
#include <cstdlib>
#include <unistd.h>

using namespace std;

#define MAX_SIZE 100
#define MAX_OPERATIONS 20

int a[MAX_SIZE];     // global array
int n = 0;           // number of elements in array
int operations = 0;

pthread_mutex_t lock;

void* thread1(void* arg) {
    while (true) {
        pthread_mutex_lock(&lock);

        if (operations >= MAX_OPERATIONS) {
            pthread_mutex_unlock(&lock);
            break;
        }

        if (n < MAX_SIZE) {
            int value = rand() % 100;
            a[n++] = value;
```

```
        }

        pthread_mutex_unlock(&lock);
        usleep(700000); // 0.7s
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    cout << "Program finished after "
         << operations << " operations." << endl;

    return 0;
}
```

```
        cout << "Thread 1: Added " << value
             << ", Number of elements: " << n << endl;
        operations++;
    }

    pthread_mutex_unlock(&lock);
    usleep(500000); // 0.5s
}
return NULL;
}

void* thread2(void* arg) {
    while (true) {
        pthread_mutex_lock(&lock);

        if (operations >= MAX_OPERATIONS) {
            pthread_mutex_unlock(&lock);
            break;
        }

        if (n > 0) {
            int removed = a[--n];
            cout << "Thread 2: Removed " << removed
                 << ", Number of elements: " << n << endl;
            operations++;
        } else {
            cout << "Thread 2: Nothing in array a" << endl;
        }
```

Explanation:

- The program creates **two concurrent threads** using POSIX threads.
- A **global array a** is shared between the threads.
- The variable n stores the current number of elements in the array.
- A **mutex** is used to synchronize access to the shared array.

Thread 1 (Producer):

- Generates random integers.
- Inserts elements into the array.
- Prints the number of elements after insertion.

Thread 2 (Consumer):

- Removes elements from the array.
- Prints the number of elements after removal.
- Prints *"Nothing in array a"* if the array is empty.

Main Thread:

- Initializes the mutex.
- Creates and joins the two threads.
- Destroys the mutex before program termination.
- The mutex prevents race conditions and ensures correct synchronization.

Step 3: Compile the program

```
yangpuck@NguyenDuongPhuc:~$ g++ exercise3.cpp -o exercise3 -pthread
```

Step 4: Run the program and observe the output

```
yangpuck@NguyenDuongPhuc:~$ ./exercise3
Thread 1: Added 83, Number of elements: 1
Thread 2: Removed 83, Number of elements: 0
Thread 1: Added 86, Number of elements: 1
Thread 2: Removed 86, Number of elements: 0
Thread 1: Added 77, Number of elements: 1
Thread 2: Removed 77, Number of elements: 0
Thread 1: Added 15, Number of elements: 1
Thread 1: Added 93, Number of elements: 2
Thread 2: Removed 93, Number of elements: 1
Thread 1: Added 35, Number of elements: 2
Thread 1: Added 86, Number of elements: 3
Thread 2: Removed 86, Number of elements: 2
Thread 1: Added 92, Number of elements: 3
Thread 2: Removed 92, Number of elements: 2
Thread 1: Added 49, Number of elements: 3
Thread 2: Removed 49, Number of elements: 2
Thread 1: Added 21, Number of elements: 3
Thread 1: Added 62, Number of elements: 4
Thread 2: Removed 62, Number of elements: 3
Thread 1: Added 27, Number of elements: 4
Program finished after 20 operations.
```